Java\com\cognizant\spring_learn\controller

HelloController.java:

```
package com.cognizant.spring_learn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    // Initialize the logger for this class
    private static final Logger LOGGER =
LoggerFactory.getLogger(HelloController.class);

    @GetMapping("/hello")
    public String sayHello() {
        // Start log
        LOGGER.info("START - sayHello()");

        String response = "Hello World!!";

        // End log
        LOGGER.info("END - sayHello()");

        return response;
    }
}
```

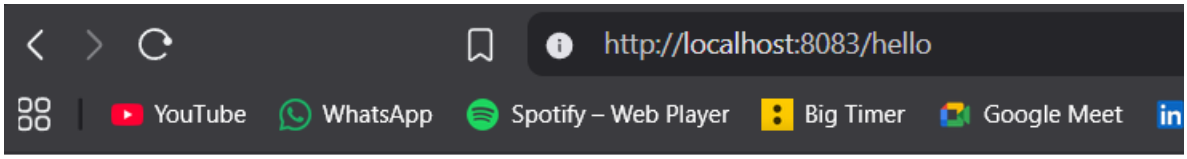
Java\com\cognizant\spring_learn

SpringLearnApplication.java:

```
package com.cognizant.spring_learn;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;

@SpringBootApplication
```

```java
public class SpringLearnApplication {

    private static final Logger LOGGER =
LoggerFactory.getLogger(SpringLearnApplication.class);

    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
        LOGGER.info("Inside main method: SpringLearnApplication started
successfully!");

        // Invoke the new method to test the XML Bean
        displayDate();
    }

    public static void displayDate() {
        LOGGER.info("START - displayDate()");

        // 1. Create the ApplicationContext pointing to the XML file
        ApplicationContext context = new ClassPathXmlApplicationContext("date-
format.xml");

        // 2. Get the dateFormat bean
        SimpleDateFormat format = context.getBean("dateFormat",
SimpleDateFormat.class);

        try {
            // 3. Parse the string and display the result
            Date date = format.parse("31/12/2018");
            System.out.println("Parsed Date: " + date);
            LOGGER.info("Parsed Date output successfully.");
        } catch (ParseException e) {
            System.out.println("Error parsing the date: " + e.getMessage());
        }

        LOGGER.info("END - displayDate()");
    }
}
```

Output:



Hello World!!

SME Walkthrough: HTTP Headers

1. Viewing HTTP Headers in the Chrome Network Tab

- Open Google Chrome and navigate to http://localhost:8083/hello.
- Press **F12** (or right-click and select **Inspect**) to open Developer Tools.
- Navigate to the **Network** tab and refresh the page (F5).
- Click on the network request named hello in the list.
- Look at the **Headers** panel on the right side. Under **Response Headers**, you will notice a key difference from your previous exercise:
    - Content-Type: text/plain;charset=UTF-8
    - Because your controller returns a plain Java String rather than an object, Spring automatically sets the Content-Type to text/plain. This instructs the browser to render the response exactly as standard text without trying to parse it as JSON or HTML.

2. Viewing HTTP Headers in Postman

- Open Postman, set the method to **GET**, and enter http://localhost:8083/hello.
- Click **Send**.
- In the lower half of the screen (the Response section), click on the **Headers** tab next to "Body".
- You will see the table of HTTP headers returned by the server.
    - Along with Content-Type: text/plain;charset=UTF-8, you will also see Content-Length: 13 (which is exactly the number of characters in "Hello World!!").

- You will also see the Date and the Keep-Alive headers that the embedded Tomcat server handles automatically for you.